

✓ Introduction + Base Array

```
import numpy as np

base_array = np.array([[1, 2, 3],[4, 5, 6]])

print(f"base_array: \n", base_array)
```

```
base_array:
[[1 2 3]
 [4 5 6]]
```

✓ Explanation

- 2 Dimensional array is created.
- It has rows (horizontal) and columns (vertical).

✓ Array Creation

```
zeros_array = np.zeros((2, 3))
print("Zeros array:\n", zeros_array)

ones_array = np.ones((2, 3))
print("Ones array:\n", ones_array)

range_array = np.arange(1, 10, 2)
print("Arange array:\n", range_array)

linear_array = np.linspace(0, 1, 5)
print("Linspace array:\n", linear_array)
```

```
Zeros array:
[[0. 0. 0.]
 [0. 0. 0.]]
Ones array:
[[1. 1. 1.]
 [1. 1. 1.]]
Arange array:
[1 3 5 7 9]
Linspace array:
[0.    0.25 0.5  0.75 1.  ]
```

✓ Explanation

- `np.zeros()` → Creates a table filled with 0
- `np.ones()` → Creates a table filled with 1
- `np.arange()` → Generates sequence with step size (1 → 3 → 5 → 7 → 9)
- `np.linspace()` → Generates evenly spaced values

✓ Array Properties

```
print(f"base_array.ndim: ", base_array.ndim)
print(f"base_array.shape: ", base_array.shape)
print(f"base_array.size: ", base_array.size)
print(f"base_array.dtype: ", base_array.dtype)
```

```
base_array.ndim: 2
base_array.shape: (2, 3)
base_array.size: 6
base_array.dtype: int64
```

✓ Explanation

- **ndim**: how many dimensions the data has
- **shape**: structure (rows, columns)
- **size**: total number of values
- **dtype**: type of data stored

✓ Indexing and Slicing

```
print(f"base_array[0, 1]: \n", base_array[0, 1])
print(f"base_array[:, 0]: \n", base_array[:, 0])
print(f"base_array[1, :]: \n", base_array[1, :])
print(f"base_array[::-1]: \n", base_array[::-1])
```

```
base_array[0, 1]:
2
base_array[:, 0]:
[1 4]
base_array[1, :]:
[4 5 6]
base_array[::-1]:
[[4 5 6]
 [1 2 3]]
```

✓ Explanation

- **base_array[0,1]**: first row, second column → value = 2
- **base_array[:,0]**: take ALL rows, first column → (1, 4)
- **base_array[1,:]**: second row, all columns → (4, 5, 6)
- **base_array[::-1]**: flips the whole table upside down

✓ Mathematical Operations

```
print(f"base_array + 10: \n", base_array + 10)
print(f"base_array * 2: \n", base_array * 2)
print(f"base_array ** 2: \n", base_array ** 2)
```

```
base_array + 10:
[[11 12 13]
 [14 15 16]]
base_array * 2:
[[ 2  4  6]
 [ 8 10 12]]
base_array ** 2:
[[ 1  4  9]
 [16 25 36]]
```

✓ Explanation

- **+ 10** → adds 10 to every value
- *** 2** → doubles every value
- **** 2** → squares every value

✓ Universal Functions

```
print(f"np.sqrt(base_array): \n", np.sqrt(base_array))
print(f"np.exp(base_array): \n", np.exp(base_array))
print(f"np.mean(base_array): \n", np.mean(base_array))
```

```
np.sqrt(base_array):
[[1.          1.41421356  1.73205081]
 [2.          2.23606798  2.44948974]]
np.exp(base_array):
[[ 2.71828183  7.3890561  20.08553692]
 [ 54.59815003 148.4131591 403.42879349]]
np.mean(base_array):
3.5
```

✓ Explanation

- `sqrt` → square root (like $\sqrt{1}$, $\sqrt{2}$, $\sqrt{3}$...)
- `exp` → exponential growth (used in biology, population, ML)
- `mean` → average value of all numbers

✓ Reshaping Data

```
reshaped = base_array.reshape(3, 2)
print(f"reshaped: \n", reshaped)
```

```
reshaped:
[[1 2]
 [3 4]
 [5 6]]
```

✓ Explanation:

- Reorganize data structure without changing values.
- From "2 samples × 3 features"
- To "3 samples × 2 features"

✓ Flattening Data

```
flat = base_array.ravel()
print(f"flat: \n", flat)
```

```
flat:
[1 2 3 4 5 6]
```

✓ Explanation

- Convert 2D table → 1D list
- Used when feeding data into machine learning models

✓ View vs Copy

```
view_data = base_array[:]
copy_data = base_array.copy()
```

```
print(f"view_data: \n", view_data)
print(f"copy_data: \n", copy_data)
```

```
view_data:
[[1 2 3]
 [4 5 6]]
copy_data:
[[1 2 3]
 [4 5 6]]
```

✓ Explanation

- View → shares memory (changes affect original)
- Copy → independent dataset

✓ Broadcasting

```
vector = np.array([10, 20, 30])

print(f"base_array + vector: \n", base_array + vector)
```

```
base_array + vector:
[[11 22 33]
 [14 25 36]]
```

✓ Explanation

- NumPy automatically “extends” smaller data to match bigger data
- NumPy repeats the small list for each row:

```
[1,2,3] + [10,20,30]
[4,5,6] + [10,20,30]
```

- Result: element-by-element addition

✓ Aggregation

```
print(f"np.sum(base_array): ", np.sum(base_array))
print(f"np.sum(base_array, axis=0): ", np.sum(base_array, axis=0))
print(f"np.sum(base_array, axis=1): ", np.sum(base_array, axis=1))
```

```
np.sum(base_array): 21
np.sum(base_array, axis=0): [ 5  7  9]
np.sum(base_array, axis=1): [ 6 15]
```

✓ Explanation

- `sum` → adds everything → total = 21
- `axis=0` → sum column by column

```
(1+4, 2+5, 3+6)
```

- `axis=1` → sum row by row

```
(1+2+3, 4+5+6)
```

✓ Filtering

```
print(f"base_array[base_array > 3]: \n", base_array[base_array > 3])

print(f"np.where(base_array > 3, base_array, 0): \n", np.where(base_array > 3, base_array, 0))

base_array[base_array > 3]:
[4 5 6]
np.where(base_array > 3, base_array, 0):
[[0 0 0]
 [4 5 6]]
```

✓ Explanation

- `base_array[base_array > 3]` → Only keep numbers bigger than 3
- `np.where(base_array > 3, base_array, 0)` → If value > 3, keep it, otherwise replace with 0

✓ Concatenation

```
a = base_array
b = base_array

print(f"np.concatenate([a, b], axis=0): \n", np.concatenate([a, b], axis=0))

np.concatenate([a, b], axis=0):
[[1 2 3]
 [4 5 6]
 [1 2 3]
 [4 5 6]]
```

✓ Explanation

- We merge datasets:
- `axis=0` → stack vertically (more samples)
- `axis=1` → add features

✓ Sorting Data

```
print(f"np.sort(base_array, axis=None): \n", np.sort(base_array, axis=None))

np.sort(base_array, axis=None):
[1 2 3 4 5 6]
```

✓ Explanation

- Arrange values in order

✓ Handling Missing Data

```
data = base_array.astype(float)
data[0, 0] = np.nan
print("Data with missing value:\n", data)
```

```
missing_mask = np.isnan(data)
print("\nMissing value mask:\n", missing_mask)
```

```
clean_data = np.nan_to_num(data, nan=0)
print(clean_data)
```

```
Data with missing value:
[[nan  2.  3.]
 [ 4.  5.  6.]]
```

```
Missing value mask:
[[ True False False]
 [False False False]]
[[0.  2.  3.]
 [4.  5.  6.]]
```

✓ Explanation

1. Convert to float (required for NaN support)
2. Introduce missing value (simulate real-world dataset issue)
3. Show updated dataset
4. Detect missing values
5. Replace NaN with 0

✓ Random Mudule

```
rng = np.random.default_rng()

print(f"rng.integers(1, 10, size=(3,3)): \n", rng.integers(1, 10, size=(3,3)))
print(f"rng.uniform(0, 1, size=(3,3)): \n", rng.uniform(0, 1, size=(3,3)))
```

```
rng.integers(1, 10, size=(3,3)):
[[5 9 4]
 [3 4 1]
 [6 6 6]]
rng.uniform(0, 1, size=(3,3)):
[[0.531269  0.49322412 0.27340735]
 [0.68779431 0.35983031 0.44124813]
 [0.24703223 0.86620554 0.49514912]]
```

✓ Explanation

- `rng = np.random.default_rng()` → This creates a random number generator machine.
- `rng.integers(1, 10, size=(3,3))` → Creates a 3×3 table of random whole numbers between 1–9.
- `rng.uniform(0, 1, size=(3,3))` → Creates random decimal numbers between 0 and 1.